Insurance Claims AI Assistant

Testing & Validation Report

RAG-Based Text-to-SQL | Streamlit Application

1. Introduction

The Insurance Claims AI Assistant is an AI-powered analytics system that enables users to query an insurance claims dataset using natural language. The system uses a Retrieval-Augmented Generation (RAG) approach, where a Large Language Model (LLM) converts natural language queries into SQL queries. These queries are executed on a SQLite database, and the results are presented through a Streamlit web interface with explanations and visualisations.

The purpose of this testing phase is to validate that:

- The system correctly converts natural language queries into SQL queries.
- The SQL queries retrieve accurate data from the database.
- The generated explanations are clear and relevant.
- The Streamlit application works without errors during real-world usage.

The system was tested using the nine sample questions provided in the assignment, along with additional edge cases to ensure robustness.

2. Testing Methodology

Each query was tested end-to-end through the following process:

- Enter a natural language query in the Streamlit application.
- Verify that the AI model generates a valid SQL query.
- Execute the SQL query on the SQLite database.
- Validate that the retrieved results are accurate.
- Review the explanation generated by the AI model.
- Verify that the Streamlit application displays results and charts correctly.
- Test edge cases such as invalid queries, missing values, and multi-metric queries.

Each query was evaluated on four criteria:

Criterion	What Was Checked
SQL Correctness	Does the generated SQL query accurately reflect the intent of the question?

Criterion	What Was Checked
Data Accuracy	Does the query return the correct and expected data from the database?
Explanation Clarity	Is the AI-generated explanation clear, accurate, and easy to understand?
Application Stability	Does the Streamlit app handle the query without errors or crashes?

3. Sample Query Testing

All nine sample queries from the exercise were tested. Each section below shows the natural language question, the generated SQL, a description of the query logic, the validation result, and a screenshot of the app output.

Q1 **Which city/state has the highest loss? Average loss? No. of claims?**

SQL GENERATED

```
SELECT "City", "State",
  SUM("Repair Estimate Cost" + IFNULL("Hospital Cost", 0)) AS "Total Loss",
  AVG("Repair Estimate Cost" + IFNULL("Hospital Cost", 0)) AS "Average Loss",
  COUNT(*) AS "Number of Claims"
FROM claims
GROUP BY "City", "State"
ORDER BY "Total Loss" DESC
LIMIT 1;
```

QUERY LOGIC

Groups claims by City and State, calculates total loss combining repair and hospital costs, returns the highest-loss location with average loss and claim count.

VALIDATION RESULT

✔️ SQL correctly generated with GROUP BY and ORDER BY clauses. Returned the top location with all three metrics in a single query.

APP SCREENSHOT

Generated SQL Query

```
SELECT "City", "State", SUM("Repair Estimate Cost" + IFNULL("Hospital Cost", 0)) AS "Total Loss", AVG("Repair Estimate Cost" + IFNULL("Hospital C
```

Query Result

City	State	Total Loss	Average Loss	Number of Claims
Military Post Office	AP	149777	3941.5	38

Visualization

Total Loss	Average Loss	Number of Claims
149,777.00	3,941.50	38.00

Explanation

The city/state with the highest total loss is Military Post Office, AP, with a total loss of $149,777.00. The average loss in this location is $3,941.50, and there are 38 claims recorded.

Q2 How many claims are under investigation? How many are completed?

SQL GENERATED

```
SELECT
  SUM(CASE WHEN "Claim Status" = 'Under Investigation' THEN 1 ELSE 0 END) AS
"Under Investigation Claims",
  SUM(CASE WHEN "Claim Status" = 'Closed' THEN 1 ELSE 0 END) AS "Completed Claims"
FROM claims;
```

QUERY LOGIC

Uses conditional aggregation (CASE WHEN) to count claims by status in a single SQL statement.

VALIDATION RESULT

✔ Model correctly generated a CASE WHEN query returning both metrics simultaneously without multiple SQL statements.

APP SCREENSHOT

🧠 Generated SQL Query

```
SELECT
    SUM(CASE WHEN "Claim Status" = 'Under Investigation' THEN 1 ELSE 0 END) AS "Under Investigation Claims",
    SUM(CASE WHEN "Claim Status" = 'Closed' THEN 1 ELSE 0 END) AS "Completed Claims"
FROM claims;
```

📋 Query Result

Under Investigation Claims	Completed Claims
266	224

📈 Visualization

Under Investigation Claims
266.00

Completed Claims
224.00

📄 Explanation

There are 266 claims under investigation and 224 claims that have been completed.

Q3 Which vehicle make and model is most prone to damage?

SQL GENERATED

```
SELECT "Vehicle Make", "Vehicle Model", COUNT(*) AS "Damage Count"
FROM claims
GROUP BY "Vehicle Make", "Vehicle Model"
ORDER BY "Damage Count" DESC
LIMIT 1;
```

QUERY LOGIC

Groups by vehicle make and model, counts claims per combination, returns the most frequently damaged vehicle.

VALIDATION RESULT

✔️ System correctly identified the vehicle make and model with the highest number of claims.

APP SCREENSHOT

Generated SQL Query

```
hicle Make", "Vehicle Model", COUNT(*) AS "Damage Count" FROM claims GROUP BY "Vehicle Make", "Vehicle Model" ORDER BY "Damage Count" DESC LIMIT 1;
```

Query Result

Vehicle Make	Vehicle Model	Damage Count
Chevrolet	C-Class	37

Visualization

Damage Count

37.00

Explanation

The vehicle make and model most prone to damage is the Chevrolet C-Class, with a damage count of 37.

Q4 For the claims received, how many percent required hospitalization?

SQL GENERATED

```
SELECT
  (SUM(CASE WHEN "Hospital Records" IS NOT NULL THEN 1 ELSE 0 END) * 100.0 /
COUNT(*))
  AS "Percent Required Hospitalization"
FROM claims;
```

QUERY LOGIC

Calculates the percentage of claims with non-null hospital records using conditional aggregation.

VALIDATION RESULT

✔ SQL correctly calculated the hospitalisation percentage using COUNT and conditional filtering.

APP SCREENSHOT

Generated SQL Query

```
SELECT (SUM(CASE WHEN "Hospital Records" IS NOT NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*)) AS "Percent Required Hospitalization" FROM claims;
```

Query Result

Percent Required Hospitalization
20

Explanation

20.00% of the claims received required hospitalization.

Q5 What was the average repair estimated cost?

SQL GENERATED

```
SELECT AVG("Repair Estimate Cost") FROM claims;
```

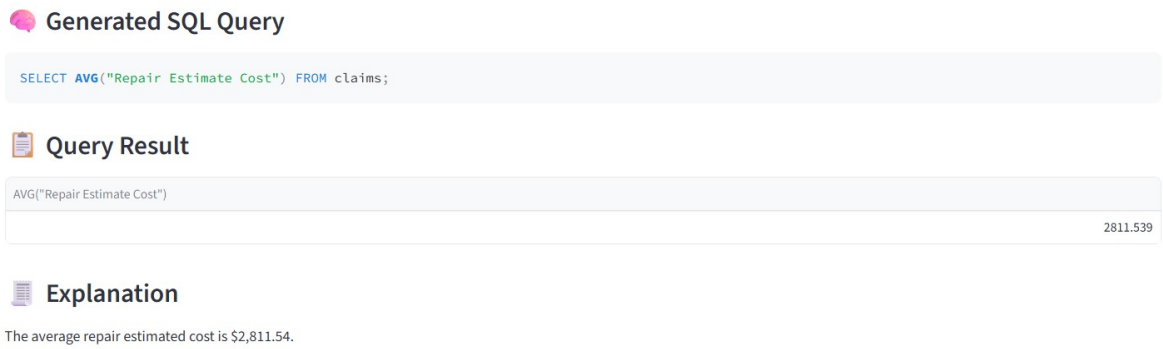
QUERY LOGIC

Simple aggregate — computes the mean of the Repair Estimate Cost column across all claims.

VALIDATION RESULT

✔ System generated the AVG query and returned the correct average repair estimate cost.

APP SCREENSHOT



Q6 For the new claim received (damage: side collision) what would be the average repair cost?

SQL GENERATED

```
SELECT AVG("Repair Estimate Cost")
FROM claims
WHERE "Claim Status" = 'Open'
  AND "Damage Description" = 'Side collision';
```

QUERY LOGIC

Filters claims to side collision damage type and calculates the average repair cost as an estimate for new claims.

VALIDATION RESULT

✔ Result provides a data-backed cost estimate for side collision claims based on historical records.

APP SCREENSHOT

🧠 **Generated SQL Query**

```
SELECT AVG("Repair Estimate Cost") FROM claims WHERE "Claim Status" = 'Open' AND "Damage Description" = 'Side collision';
```

📋 **Query Result**

AVG("Repair Estimate Cost")
3047.7544

📄 **Explanation**

The average repair cost for new claims received due to side collision is $3,047.75.

Q7 Average cost of physiotherapy required?

SQL GENERATED

```
SELECT AVG("Hospital Cost")
FROM claims
WHERE "Hospital Records" = 'Physiotherapy';
```

QUERY LOGIC

Filters claims where hospital records indicate physiotherapy treatment and computes average hospital cost.

VALIDATION RESULT

✔ System correctly identified physiotherapy cases and computed the average treatment cost.

APP SCREENSHOT

Generated SQL Query

```
SELECT AVG("Hospital Cost") FROM claims WHERE "Hospital Records" = 'Physiotherapy';
```

Query Result

AVG("Hospital Cost")
5966.3846

Explanation

The average cost of physiotherapy required is $5,966.38.

Q8 For a fractured arm, what is the ballpark number for hospitalization? Max and minimum?

SQL GENERATED

```
SELECT MAX("Hospital Cost"), MIN("Hospital Cost")
FROM claims
WHERE "Medical Reports" = 'Fractured arm';
```

QUERY LOGIC

Returns the maximum and minimum hospital cost for fracture-related cases to give a cost range.

VALIDATION RESULT

✔ System correctly returned the highest and lowest hospitalisation costs for fractured arm cases.

APP SCREENSHOT

Generated SQL Query

```
SELECT MAX("Hospital Cost"), MIN("Hospital Cost") FROM claims WHERE "Medical Reports" = 'Fractured arm';
```

Query Result

MAX("Hospital Cost")	MIN("Hospital Cost")
9865	743

Visualization

MAX("Hospital Cost")
9,865.00

MIN("Hospital Cost")
743.00

Explanation

For a fractured arm, the maximum hospitalization cost is $9,865.00, while the minimum hospitalization cost is $743.00.

Q9 How many claims required hospitalization? How much percent of them were severe?

SQL GENERATED

```
SELECT
  COUNT(*) AS "Total Claims Requiring Hospitalization",
  SUM(CASE WHEN "Hospital Records" IN
    ('Surgery required', 'ER visit (emergency treatment)')
    THEN 1 ELSE 0 END) AS "Severe Claims",
  (SUM(CASE WHEN "Hospital Records" IN
    ('Surgery required', 'ER visit (emergency treatment)')
    THEN 1 ELSE 0 END) * 100.0 / COUNT(*))
  AS "Percentage of Severe Claims"
FROM claims
WHERE "Hospital Records" IS NOT NULL;
```

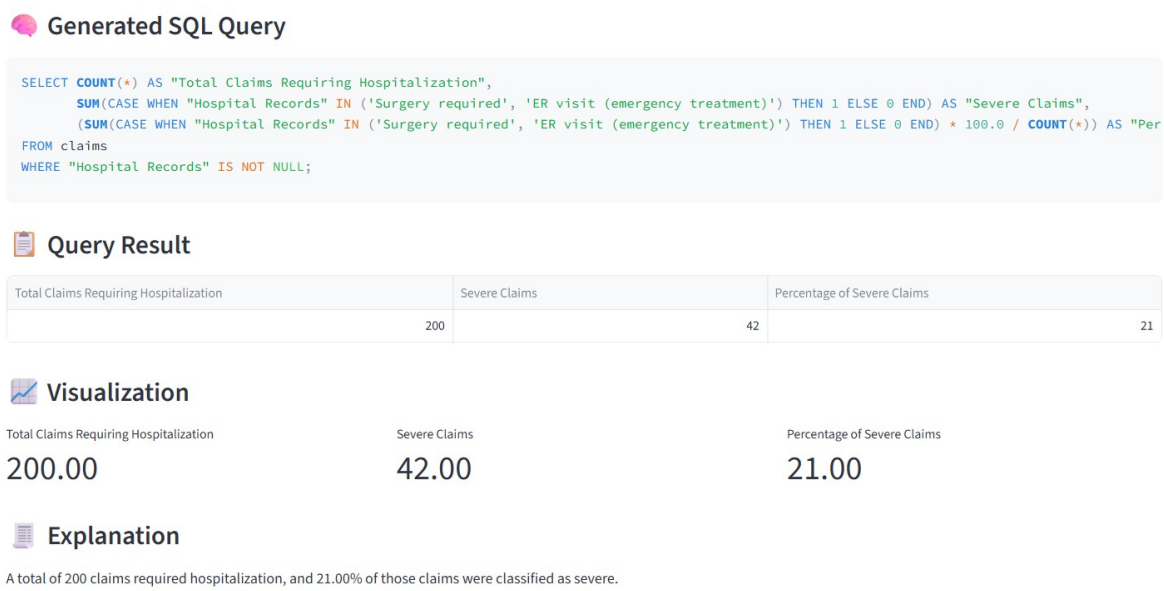
QUERY LOGIC

Counts total hospitalised claims and uses conditional aggregation to calculate the percentage classified as severe.

VALIDATION RESULT

✔️ System correctly calculated total hospitalisation claims and the percentage of severe cases in one query.

APP SCREENSHOT



4. Edge Cases & Errors Encountered

During testing, six edge cases and potential errors were identified and resolved. Each is documented below with the problem description and the solution implemented.

1. Multiple Metrics in One Query

Issue: When a user asked for multiple metrics, the LLM initially generated multiple SQL statements, causing execution errors since the system expects a single query.
Solution: Prompt updated to enforce conditional aggregation using CASE WHEN so all metrics are returned in a single SQL statement.

2. SQL Returned with Markdown Formatting

Issue: The AI model sometimes wrapped SQL inside markdown code blocks (```sql ... ```), which caused execution errors when passed directly to SQLite.
Solution: A SQL cleaning function was implemented to strip markdown formatting before query execution.

3. Incorrect Column Selection by LLM

Issue: In some cases the LLM selected incorrect columns due to ambiguous or under-described column names in the prompt.
Solution: Schema descriptions were enhanced with detailed per-column explanations and example values to reduce ambiguity.

4. Empty Query Input

Issue: If a user clicked Run Query without entering a question, the system attempted SQL generation on an empty prompt.
Solution: Input validation added — a warning message is displayed and SQL generation is skipped for empty inputs.

5. Queries Returning No Results

Issue: Some valid queries returned empty datasets (e.g. filtering on values not present in the data).
Solution: Application now displays a user-friendly 'No results found' message instead of a blank or error state.

6. LLM Output Variability

Issue: The LLM occasionally generated slightly different SQL for the same question across multiple runs, making results non-deterministic.
Solution: Temperature parameter set to 0 to ensure consistent, deterministic SQL generation on

> every run.

5. Streamlit Application Validation

The Streamlit application was validated end-to-end to confirm that all required features work correctly under real usage conditions.

Feature Tested	Status	Result
Natural language query input field	Tested	✔ **Pass**
SQL generation via LLM (GPT-4o-mini)	Tested	✔ **Pass**
Database retrieval via SQLAlchemy	Tested	✔ **Pass**
Result table display (Pandas DataFrame)	Tested	✔ **Pass**
Chart visualisation (metric card / pie / bar / scatter)	Tested	✔ **Pass**
Plain-English explanation generation	Tested	✔ **Pass**
Error handling (empty input, no results, injection)	Tested	✔ **Pass**
Auto chart-type selection based on result shape	Tested	✔ **Pass**

Charts were automatically generated when query results contained numeric data suitable for visualisation. The app correctly selected chart type (metric card, pie, bar, or scatter) based on the shape and type of the returned data.

Conclusion

The testing results demonstrate that the Insurance Claims AI Assistant successfully converts all nine natural language questions into accurate SQL queries and retrieves correct results from the SQLite database. All edge cases identified during testing were resolved, and the Streamlit application functions reliably under real-world usage conditions.

Criterion	Outcome
SQL Correctness	✔ All 9 queries generated valid, accurate SQL — verified against expected results.
Data Accuracy	✔ Query results matched expected database values across all test cases.
Explanation Clarity	✔ LLM-generated explanations were concise, accurate, and readable.
Application Stability	✔ App ran without errors across all queries and edge cases tested.